
Compiler-Guided Autoformalization with Adaptive Multi-View Representations

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Autoformalization for verifiable coding requires a source-grounded interpretation
2 and a computational route for checking it. We describe a compiler-guided archi-
3 tecture that couples engineered symbolic intermediate representations (IRs) to
4 adaptive multi-view representation learning, and we report the automated struc-
5 tural, native-training, and checker evidence that was actually obtained. Pipeline
6 comparisons A–E vary the source-to-proof mechanism. Learning comparisons
7 T0–T5 vary sample memory, shared parameters, isolated proof heads, promoted
8 guidance, and compiler repair. Those two matrices answer different questions and
9 are not merged.

10 We report automated structural correctness, exact-goal native checker receipts,
11 teacher-agreement diagnostics, learned-update records, and finite constructed con-
12 formance. Independent human source-facet fidelity and agreement were not col-
13 lected. Checker and teacher success are not original-source semantic gold. On
14 the locked 1913-unit final-test population, matched A–E coverage and cost re-
15 main unrun, independent fidelity is unmeasured, and native useful-proof transfer
16 is unavailable on the sealed checker PATH. T1 remains a sample-memory diagnos-
17 tic. Across three seeds, T2 completed shared-parameter updates and T3 applied
18 compiler-structural checker feedback. Perfect T2 reconstruction is target-assisted,
19 not source-free generalization. Actual fixed-canary promotion returned no candi-
20 date; T4 and Arm E remain unactivated; T5 has no measured held-out improve-
21 ment. A synthetic policy–code–trace example and twelve reconstructed finite se-
22 mantic checks illustrate composition and are not natural-distribution accuracy.

1 Introduction

24 Autoformalization makes heterogeneous material available to mechanical reasoning, but a formula
25 can be well typed and provable while misrepresenting its source [Wu et al., 2022, Azerbayev et al.,
26 2023, Jiang et al., 2024]. A requirement, an implementation, and an execution trace also have dif-
27 ferent truth conditions: an obligation is not an observation, permission is not execution, and belief
28 is not fact. Mathematical autoformalization demonstrates useful natural-language translation into
29 formal statements. The coding-verification question is how to learn better source-to-symbolic map-
30 pings while preserving those distinctions, without treating retrieval, planning, or a solver candidate
31 as a proof.

32 The method couples a deterministic semantic pipeline to adaptive representation learning. Compilers
33 create source-grounded IRs, decomposers expose multiple logical views, and decompilers recon-
34 struct inspectable content. An adaptive modal autoencoder learns reusable feature-to-embedding
35 and feature-to-family/view parameters against teacher targets. Reconstruction, structural, prove-
36 nance, grammar, and cross-view diagnostics identify where the symbolic pipeline is inadequate.

37 Bounded optimization then either updates shared parameters or proposes a separately validated com-
38 piler/decompiler change. Checked proof feedback, when admitted, trains isolated prediction heads
39 rather than rewarding arbitrary changes that make the input easier to prove.

40 This is neither one end-to-end language model nor unsupervised discovery of a universal logic. The
41 schema, operators, sorts, and admissibility rules are engineered. Learning estimates content, asso-
42 ciations, rankings, and repair priorities within those contracts. Several implemented paths coexist:
43 deterministic compilation, adaptive diagnostic learning, packed tensor updates, source-withheld re-
44 construction, and bounded model-assisted synthesis. We distinguish their inputs, outputs, objectives,
45 and activation.

46 The original draft asked whether shared learned representations and checked feedback improve inde-
47 pendently judged source fidelity and useful proof coverage on unseen material. Independent human
48 labels were not collected, so that question remains unmeasured. The measured contribution of this
49 manuscript is narrower:

- 50 1. a source-grounded account of compilation, multi-view IRs, and adaptive learning, with
51 equations audited for reconstruction versus text, source maps versus truth, planning versus
52 proof, and implemented versus executed tense;
- 53 2. matched pipeline conditions A–E and learning conditions T0–T5, reported separately, with
54 every primary final-test cell retained as unrun, unavailable, or unmeasured rather than as a
55 silent zero;
- 56 3. finite constructed conformance, solver-local policy–code–trace witnesses, development
57 exact-goal checker receipts, and native-training diagnostics, each labeled by population
58 and not promoted to held-out source-semantic gold.

59 Historical PDF recovery text is retained under pages/ as provenance. That recovery copy still
60 contains unmeasured promises; this manuscript does not present those promises as results.

61 2 Representations, teachers, and reconstruction

62 2.1 What is shared, and what is learned

63 Three axes remain distinct. Source format names prose, code, traces, tables, or extracted media.
64 Domain names Legal, Security, Intent, software, or UI declarations. Logical view names deontic,
65 temporal, frame, first-order, cognitive/event, graph, or proof-oriented structure. Sharing canonical
66 records does not equate domain ontologies or logical interpretations.

67 The domain-neutral kernel supplies canonical bytes, identities, provenance, source spans, assump-
68 tions, claims, obligations, and result protocols. Domain adapters retain their own semantic slots.
69 A symbolic declaration is discrete and inspectable; its content identifier binds an encoding under a
70 declared profile, not a learned location in a vector space. Equal vectors or nearby embeddings do not
71 imply equivalent formulas. Six intermediate objects have different roles: canonical symbolic decla-
72 rations, family-specific formulas, learned numerical features, graph relationships, search plans, and
73 proof/evidence records. A search plan is not a formula; a declaration is not its verification result;
74 and a vector is not a proof.

75 2.2 Supervisory targets are teachers, not source gold

76 The concrete legal training record contains source identity, normalized text, an embedding-model
77 identifier and vector, a modal IR document, frame candidates, parser trace, and losses. The default
78 convenience embedding is a deterministic mock (mock:stable-sha256); a local codec can supply
79 spaCy-derived feature hashing; externally supplied vectors carry their producer identity. Modal-
80 family targets can be derived from parsed operators; view targets come from bridge contracts; frames
81 can be retrieval-selected; proof-feedback labels require admitted verifier records. Reproducing these
82 labels measures distillation of those teachers. Independent annotations are required to measure
83 fidelity to the original source, and those annotations were not collected.

84 For a source x , write $I = C_\psi(x)$ for the symbolic compiler output, $e = T(x)$ for the de-
85 clared embedding target, and $f = F(x, I)$ for reusable features. Features include lexical cues,

86 actor/action/object roles, conditions and exceptions, temporal structure, predicate arity, quantifier
 87 scope, provenance, frame relations, compiler contracts, and reconstruction diagnostics. The adap-
 88 tive model predicts an embedding $\hat{e}$, a modal-family distribution p_F , and a view distribution p_V .
 89 Some features depend on an existing parse, so this is often parser-assisted learning, not a demon-
 90 strated raw-text-only compiler.

91 2.3 Compilers and two meanings of reconstruction

92 The local modal codec uses deterministic registry cues, spaCy-derived features, BM25 frame
 93 grounding, and frame-logic consistency checks. A selected structured-text profile has a seven-field
 94 rule—modality, actor, action, object, conditions, exceptions, and temporal qualifiers—and explic-
 95 itly disables learned guidance and repair. Formalization adapters produce named formula views and
 96 source maps. The maps bind IR constituents to source evidence; they are not proof of a correct
 97 interpretation. Media extraction is a limitation: no OCR/ASR theorem performance is claimed.

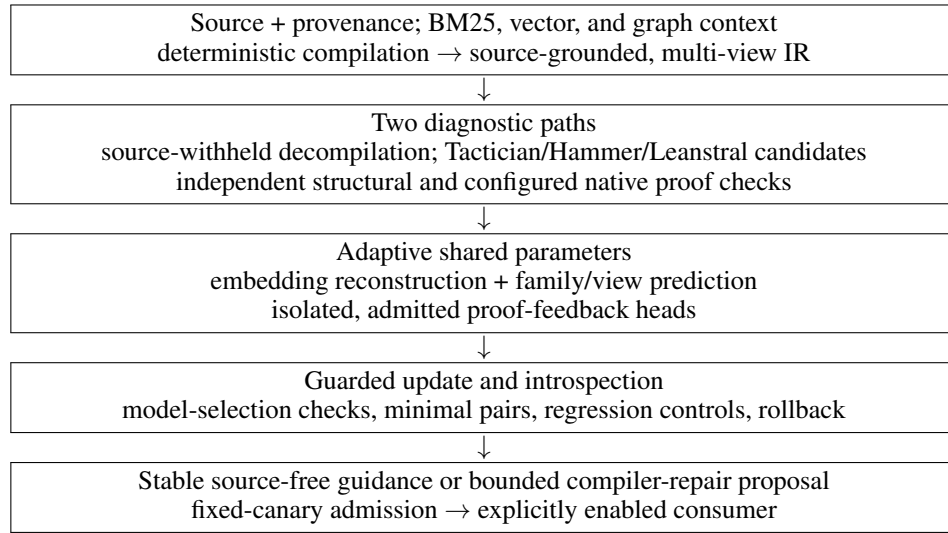


Figure 1: Coupled compilation, retrieval, proof assistance, and learning. Arrows show permissible information flow, not one automatically activated entry point. Only admitted feedback trains proof heads; the frozen canonical compiler remains deterministic.

98 Symbolic reconstruction follows

$$I_1 = C_\psi(x), \quad \hat{x} = D_\omega(I_1), \quad I_2 = C_\psi(\hat{x}). \quad (1)$$

99 A decompiler renders roles, predicates, modality, conditions, exceptions, and temporal structure into
 100 inspectable text. Source-withheld evaluation gives the realizer the IR, not the originating source,
 101 locator, source map, private parser state, or gold target. Comparing I_1 and I_2 tests reconstructable
 102 facets and compiler/realizer consistency. It does not prove source fidelity: both programs could
 103 consistently omit the same exception. Forward (gold→first IR), cycle (first IR→second IR), and
 104 end-to-end (gold→second IR) scores remain distinct.

105 The autoencoder’s numerical reconstruction is different. Encode builds a state containing family
 106 predictions and an embedding projection; decode returns that projection as a numeric vector. Its
 107 basic reconstruction target is the sample embedding, not arbitrary source prose. Calling this vector
 108 reconstruction a trained free-text decoder would misstate the implementation: numerical decode is
 109 not a trained free-text decoder. The evaluated T2 path additionally receives the target embedding and
 110 may return it directly, so its perfect reconstruction score is not a source-free decoder measurement.

3 Adaptive learning and assistance

3.1 Shared parameters versus sample memory

The adaptive implementation has sparse parameter groups for feature embeddings, modal families, semantic slots, predicate arguments, logic signatures, view logits, decompiler plans, and family-slot-view interactions. It also retains compatibility fields indexed by sample identity. The generalizable training route disables per-sample memory for updates and evaluation. Reconstructing a stored vector for a known sample is not generalization. A schematic of the shared heads is

$$\hat{e} = e_0(f) + \sum_g A_g(f)W_g, \quad (2)$$

$$p_F = \text{softmax}\left(\ell_F^0(f) + \sum_g A_g(f)U_g\right), \quad p_V = \text{softmax}\left(\ell_V^0(f) + \sum_g A_g(f)V_g\right). \quad (3)$$

Here A_g is activity of a bounded feature group, and W_g, U_g, V_g are enabled embedding and logit tables. This notation summarizes the packed implementation; it is not a Transformer or variational autoencoder. A forward-only no-op is reported separately from a parameter update.

3.2 Packed objective and isolated proof heads

The packed tensor path gathers active rows, constructs predictions, reduces losses in FP32, back-propagates, clips gradients, applies an SGD-style step, and scatters changed rows into transaction-aware state. The inspected update is not Adam. For the active heads,

$$L_{\text{packed}} = H(q_F, p_F) + H(q_V, p_V) + \text{MSE}(e, \hat{e}) + \lambda_c(1 - \cos(e, \hat{e})) + L_{\text{norm}} + \lambda_2 R(\Theta). \quad (4)$$

Targets are normalized distributions; masks exclude unavailable rows. Neither cosine similarity nor cross-entropy is a theorem about recovered meaning. Appendix B records the globally normalized masked CE and the SGD-style step actually audited.

Seven explicit heads can predict obligation family, semantic slots, premise family, route availability, trusted outcome, reconstruction success, and minimal failing contract. Their declared weight in the primary representation objective is zero. A prediction of proof success can prioritize a route but cannot certify a formula.

Actual native export, promotion, ingestion, and default reset were run sequentially for all three T3 checkpoints on the fixed 38 canaries. All promotions returned `no_candidate`; ingestion records an identity but the default compiler does not apply learned features. Missing proof receipts, view-family weights, resolved contracts, and qualifying guidance records block activation. Empty family-metric maps supply no empirical guardrail score. Default reset is not rollback of an applied learned effect; T4 and E remain unavailable.

3.3 Retrieval, planning, and proof proposals

BM25, vector indexes, and knowledge-graph traversal supply complementary context rather than truth criteria. Index construction is not encoder training. Context becomes a premise only after grounding, applicability, and policy checks; absence from retrieval is not logical negation.

The Tactician deterministically orders caller-provided sources under an explicit policy and creates an acyclic subgoal plan. Planning performs no proof. The ITP Hammer selects premises, lowers a supported first-order-safe fragment, runs an allowed solver portfolio, and attempts native reconstruction. Solver output is a candidate until reconstruction is accepted by the originating checker. Leanstrat supplies advisory proof-engineering proposals; a model response does not replace checking. Generated Lean containing `sorry`, `admit`, or unapproved axioms is rejected before any kernel claim. These mechanisms are implemented routes. On the sealed validation PATH, native Lean/Z3/CVC5/Vampire/E/Coq/Isabelle assistance was unavailable, and no Hammer or Leanstrat benchmark from other papers was run.

4 Multi-view translation and a coding example

4.1 Profile-indexed semantics and a conditional transfer rule

Each logical view has a signature, sentence language, semantic models, satisfaction relation, and a supported query fragment. For illustration, a normal modality can be interpreted with an indexed accessibility relation

$$\llbracket \Box_i \phi \rrbracket(w) = \forall v [R_i(w, v) \Rightarrow \llbracket \phi \rrbracket(v)]. \quad (5)$$

Equation 5 is an illustrative semantic recipe, not a claim that every repository view is compiled by it. Ordinary normal modal logic is not a universal deontic solution [Benzmüller et al., 2020]. $O(Gp)$ and $G(Op)$ are not equated without a profile-specific commutation result.

A translation record must state source/target views, profile, signature map, assumptions, supported constructs, named losses, preserved property, and checker evidence. Translation validation examines the produced transformation rather than trusting every compiler invocation [Pnueli et al., 1998]. A legacy TDFOL-to-FOL converter that erases unary temporal and deontic operators and replaces strong until by conjunction is a negative control; a resulting solver success is not a valid modal proof.

Let $\Gamma \models_{L_s} \phi$ be the desired source-view consequence. A translator τ maps premises and goal into L_t . Under bridge assumptions B , let $R_B(s, t)$ relate admitted source and target models. The required directionality is

$$s \models \Gamma \wedge R_B(s, t) \Rightarrow t \models \tau(\Gamma), \quad (6)$$

$$t \models \tau(\phi) \wedge R_B(s, t) \Rightarrow s \models \phi. \quad (7)$$

For every admitted source model satisfying Γ and B there must be a related target model; otherwise the relation can be vacuous. If a target checker soundly establishes $\tau(\Gamma) \models \tau(\phi)$, these conditions transfer the consequence back. This is a *conditional paper proposition* to discharge per supported translation, not a machine-checked soundness theorem about every implementation translator. Inconsistent premises are refused rather than exploded into an accepted transfer.

4.2 Synthetic policy–code–trace illustration

Consider a synthetic coding requirement: “Protected resources must not be written without approval. Each write must be audited within two steps.” It is a specification example, not legal advice and not a production experiment. One source supplies the requirement, a second supplies implementation text, and a third supplies observations. The normative view may represent the first sentence as

$$\text{Protected}(r) \wedge \neg \text{Approved}(a, r, t) \Rightarrow O_{a,t,\theta}(\neg \text{Write}(a, r, t)). \quad (8)$$

The policy-to-compliance bridge defines which observed behaviors count as violations. It does not derive actual non-writing from the obligation. The compliance queries are

$$Q_1 = \forall a, r, t : \text{Protected}(r) \wedge \neg \text{Approved}(a, r, t) \Rightarrow \neg \text{Write}(a, r, t), \quad (9)$$

$$Q_2 = \forall a, r, t : \text{Write}(a, r, t) \Rightarrow \exists u \in [t, t + 2] : \text{Audit}(a, r, u). \quad (10)$$

A deterministic implementation model $\text{Write} = \neg \text{Protected} \vee \text{Approved}$ makes Q_1 hold on that finite Boolean model. An always-write implementation supplies a countermodel when the resource is protected and approval is false. Transferring either result to actual source code requires a justified extraction or refinement relation; finding the word “approve” in a function name is insufficient. For Q_2 , a write at step 0 and a matching audit at step 1 satisfy the bounded requirement for that observed event; an audit at step 3 violates it only when capture is complete through the deadline; an incomplete window yields unknown; a same-looking name in a different tenant cannot be joined without an admitted identity mapping. These are finite constructed witnesses, not compiler soundness and not Table 3.

5 Evaluation protocol

Pipeline arms A–E and learning arms T0–T5 share frozen source-family/time grouping and a 1913-unit final-test lock when a comparison calls for the same population, but they answer different questions. A–E vary the source-to-proof pipeline under a matched checker contract. T0–T5 vary learning and transfer from a sealed starting representation.

Table 1: Pipeline arms A–E vary source-to-proof mechanism. They are not the T0–T5 learning matrix.

Arm	Mechanism (matched source tasks, vocabulary, checker, and budget where required)
A	Direct model-to-target formalization.
B	Deterministic bounded compiler; frozen canonical profile; explicit abstention; learned guidance disabled.
C	Typed source-grounded multiview without admitted proof transfer; cannot consume D bridge evidence.
D	C plus checked property-specific bridges and compatible premise assembly.
E	D plus bounded learned view/premise advice; checker unchanged; gated to actual loaded-guidance identity.

Table 2: Learning arms T0–T5 vary memory, shared parameters, isolated feedback, promotion, and repair. They are not the A–E pipeline matrix.

Arm	Learning comparison
T0	Deterministic/codec baseline; no shared-parameter fit.
T1	Adaptive with sample memory (seen-source diagnostic, not a fair generalization condition).
T2	Shared-parameter-only updates; sample memory disabled.
T3	Isolated trusted proof heads from a matched T2 checkpoint.
T4	Source-free promoted guidance actually loaded by a compiler/realizer consumer.
T5	Bounded executable compiler/decompiler repair from a sealed starting arm.

Prespecified seeds are 104729, 130363, and 155921. Frozen splits are train 69 / selection 15 / fixed canary 38 / final-test 1913 locked across 20 source-family/time groups (CFR 993, MPEP 738, U.S. Code 175, guidance 7). Final-test identities, labels, maps, and outcomes do not enter training, selection, canaries, or inspection. Uncertainty, when computed, is a 95% source-family/time-group cluster bootstrap with 10000 resamples; it is not imputed for unrun cells.

Empirical scope. We evaluate automated structural and execution properties of the inspected pipeline: typed elaboration, declared compiler contracts, reconstruction agreement labeled by its producer, training and checkpoint updates with exact split/seed/cost bindings, and native checker acceptance of an exact declared formal goal under a pinned checker identity. Each reported quantity must carry its own execution record, population, seed, source, goal, premises, checker and resource bindings. A checker-accepted formal statement is not evidence that the statement faithfully captures the original natural-language source. Automatic teacher agreement and round-trip agreement are reported as such and are not original-source semantic gold.

Supported automated denominators. Constructed finite conformance is retained on its predefined witness set (AF-006 twelve reconstructed checks; AF-015 property-specific Boolean/trace cases). Native exact-goal checking is retained only where an independent receipt names the goal, premises, checker digest and outcome, including explicit failures (AF-027 Lean identity success, Lean False:=trivial type error, and Z3 unsat sentinel; sealed-PATH AF-018 probes remain unavailable). Learned updates and teacher diagnostics remain bound to the populations actually used (AF-011: 69 train plus 15 selection CFR windows, mock embeddings, claim_admissible=false; AF-009 packed-CPU numerics are a separate host diagnostic). Final-test Tables 6/11/13 keep the 1913-unit no-run denominator; unrun and unavailable are not measured zeros. Arm E stays gated to later AF-013 canary, consumer-load and rollback receipts. Development, fixed-canary and constructed results must not be presented as untouched final-distribution performance.

Unperformed human evaluation. No independent human semantic-fidelity study, inter-annotator agreement study, or adjudication was performed for this manuscript. Their proposed protocols are retained as unexecuted work and supply no empirical score. The prepared 100-unit candidate-blind sample across 20 groups remains blank; zero agreement is an inventory of missing records, not a reliability statistic. Any optional author comments, if supplied, are descriptive non-independent

Table 3: Matched A–E pipeline comparison on the locked 1913-unit final-test population. Unrun and unavailable are not measured zeros. Independent source-semantic fidelity is unmeasured.

Arm	Coverage / abstention	Independent fidelity / uncertainty	Native-checked proof / transfer	useful	Total cost / latency
A	unrun 0/1913 measured (not a zero-score success)	unmeasured (no independent gold; not a 0% or 0.0 interval)	unavailable (no native checker on sealed PATH; not 0/1913 soundness)		unrun (final-test not executed; not 0 s; human review unmeasured)
B	unrun 0/1913 measured (not a zero-score success)	unmeasured (no independent gold; not a 0% or 0.0 interval)	unavailable (no native checker on sealed PATH; not 0/1913 soundness)		unrun (final-test not executed; not 0 s; human review unmeasured)
C	unrun 0/1913 measured (not a zero-score success)	unmeasured (no independent gold; not a 0% or 0.0 interval)	unavailable (no native checker on sealed PATH; not 0/1913 soundness)		unrun (final-test not executed; not 0 s; human review unmeasured)
D	unrun 0/1913 measured (not a zero-score success)	unmeasured (no independent gold; not a 0% or 0.0 interval)	unavailable (no native checker on sealed PATH; not 0/1913 soundness)		unrun (final-test not executed; not 0 s; human review unmeasured)
E	unrun 0/1913 measured (not a zero-score success)	unmeasured (no independent gold; not a 0% or 0.0 interval)	unavailable (no native checker on sealed PATH; not 0/1913 soundness)		unrun (final-test not executed; not 0 s; human review unmeasured)

226 feedback with author provenance, remain outside gold and scored metrics, and are not required for
 227 manuscript or task completion. The manuscript’s supported contribution is limited to the automated
 228 evidence actually reported; source-semantic fidelity and its transfer remain unmeasured.

229 6 Measured results

230 6.1 A–E pipeline comparison is not a T0–T5 result

231 Table 3 reports the locked final-test A–E matrix. Unrun and unavailable are not measured zeros. In-
 232 dependent source-semantic fidelity is unmeasured and is not prover or teacher agreement. Selection-
 233 split structural counts (A unavailable 15/15; B abstained 4 and measured 11; C measured 15/15; D
 234 unsupported 15/15; E unavailable 15/15) are a separate non-Table 3 diagnostic and do not fill the
 235 1913-unit cells.

236 6.2 T0–T5 learning comparison is not an A–E result

237 Table 4 reports T0–T5. Split counts are frozen inventory. Held-out independent fidelity is unmea-
 238 sured; teacher cosine and reconstruction do not fill that column. Proof/route benefit and final-test
 239 cost remain unrun or unavailable.

240 The completed AF-029 run used MiniLM targets for all 69 train and 15 selection units. T0 com-
 241 pleted three deterministic no-update replays; T1 performed 138 sample-memory updates per seed
 242 and remains a memory diagnostic. For each seed, T2 accepted one epoch, executed five packed up-
 243 date families, and changed 182,500,992 shared numerical parameters with sample memory disabled.
 244 Earlier mock-teacher/no-update attempts are preserved as history rather than the final training result.

245 T2 evaluation returns cosine 1 and MSE 0 through a target-assisted projection that may return the
 246 supplied target embedding. Those values are not evidence of learned source-free reconstruction,
 247 generalization, or fidelity; family cross-entropy is unchanged from T0. T3 applied 256 of 2606
 248 eligible compiler-structural feedback labels per seed while preserving protected parameters. Its
 249 training-label calibration is a diagnostic, not held-out route efficacy or semantic gold. Saved T2/T3
 250 states passed a separate retained-byte checker after the original full checker exhausted memory; the
 251 original container remains failed. Native artifact transport subsequently preserved all selected bytes.
 252 T5 remains a constructed compiler-repair observation, with no measured held-out improvement.

253 6.3 Assistance comparisons remain constructed or unavailable

254 Table 5 retains the six prespecified assistance comparisons. Constructed automatic-label diagnostics
 255 do not fill natural held-out cells. Unavailable native checkers are not measured zeros. Independ-
 256 ently judged natural relevance was not collected. Selector weights are disclosed as hand-authored
 257 (trained=false); import-overlap is a proxy, not usefulness gold. Plans replay deterministically

Table 4: T0–T5 learning comparisons. Split counts are frozen inventory. Held-out independent fidelity is unmeasured. Proof/route benefit and final-test cost remain unrun or unavailable.

Arm	Source/split counts	Held-out fidelity	Proof/route benefit	Total cost
T0	train 69 / selection 15 / canary 38 / final-test 1913 locked	unmeasured (independent gold absent; teacher cosine is not this cell)	unrun on 1913 final-test units; teacher diagnostics are not route benefit	unrun on 1913 final-test units (AF-011/AF-020/AF-029 costs are not this cell)
T1	train 69 / selection 15 / canary 38 / final-test 1913 locked	unmeasured (independent gold absent; teacher cosine is not this cell)	unrun on 1913; T1 is a memorization diagnostic, not proof/route benefit	unrun on 1913 final-test units (AF-011/AF-020/AF-029 costs are not this cell)
T2	train 69 / selection 15 / canary 38 / final-test 1913 locked	unmeasured (independent gold absent; teacher cosine is not this cell)	unrun on 1913; no shared-learner native-proof gain is admitted	unrun on 1913 final-test units (AF-011/AF-020/AF-029 costs are not this cell)
T3	train 69 / selection 15 / canary 38 / final-test 1913 locked	unmeasured (independent gold absent; teacher cosine is not this cell)	unrun on 1913; 256 structural-feedback updates per seed are not final route benefit	unrun on 1913 final-test units (AF-011/AF-020/AF-029 costs are not this cell)
T4	train 69 / selection 15 / canary 38 / final-test 1913 locked	unmeasured (independent gold absent; teacher cosine is not this cell)	unavailable (T4 unactivated; consumer load is not applied learning)	unrun on 1913 final-test units (AF-011/AF-020/AF-029 costs are not this cell)
T5	train 69 / selection 15 / canary 38 / final-test 1913 locked	unmeasured (independent gold absent; teacher cosine is not this cell)	unrun (no measured held-out improvement; constructed compile is not this cell)	unrun on 1913 final-test units (AF-011/AF-020/AF-029 costs are not this cell)

Table 5: Retrieval, planning, and proof-assistance comparisons. Constructed automatic-label diagnostics do not fill natural held-out cells. Unavailable native checkers are not measured zeros.

Comparison	Controlled variable	Required outcome	Audited result / status
Lexical / vector / graph	Same constructed corpus, queries, target interpretation and budget	Independently judged relevant context and accepted-premise yield	constructed 34 docs x 12 queries; FAISS/MiniLM unavailable; automatic citation labels; does not fill natural cells
Baseline / graph selector	Same goal and candidate corpus	Recall against stated labels; proxy status; actual useful proof coverage	hand-authored weights trained=false; proxy import-overlap is a separate metric; useful native-proof coverage unavailable
Unguided / guided plan	Same admitted sources and route budget	Deterministic replay, subgoal coverage, abstention and total cost	72 constructed planning observations; executed_proofs=0; LLM nomination unavailable
Hammer / Leanstrat assistance	Same source goal, native checker and resource envelope	Candidate rate, reconstruction acceptance, unsupported and time-out rates	sealed-PATH Lean, Z3, CVC5, Vampire, E, Coq, Isabelle unavailable; native_checked_proofs=0; sorry/admit rejected before any kernel claim
Proof heads off / on	Same representation checkpoint and isolated update policy	Calibration, route value, protected-objective invariance	T3 applied 256 structural-feedback updates per seed; held-out calibration and route benefit unrun
Guidance off / promoted	Same compiler profile and frozen source families	Actual consumer activation, independent source fidelity, regressions	T4 unactivated/unavailable; e_locked; matching ingestion digest is not applied learning

258 and are not executed proofs (executed_proofs= 0). Hammer/Leanstrat native reconstruction is
 259 unavailable. No miniF2F, ProofNet, LeanDojo, CompCert, seL4, or Dafny benchmark was run.

260 6.4 Constructed conformance and development checkers

261 Twelve reconstructed finite semantic checks matched their declared expected outcomes, including
 262 26 disagreements between strong-until and conjunction among 64 length-three traces. This is a new
 263 reconstruction, not recovery of a historical Table 4 execution, and not natural-distribution accuracy.

264 On the synthetic policy–code–trace fragment, a guarded Boolean model accepts Q_1 transfer and an
 265 unguarded always-write model supplies a Q_1 countermodel. Timely versus delayed audit, incom-

plete windows, wrong-tenant identity, inconsistent premises, unjustified name extraction, obligation-not-actuality, and lossy TDFOL erasure have distinct outcomes. Native Lean Q_1 is unavailable on the sealed PATH. Zero observed false transfers on that finite set is not universal soundness. Proposition C is not represented as machine-checked compiler soundness.

Development exact-goal receipts include a Lean identity success, a Lean `False:=trivial` type error, and a Z3 unsat sentinel. Those receipts do not receive Table 3 credit.

6.5 Costs, missingness, and hypotheses

The historical AF-020 source-usage ledger totals 481.586 elapsed seconds; its phase summaries are the same intervals and are not added again. The later common training profile used 2 CPUs, 64 GiB, a 10800 s complete-T2 limit per seed, and 36000 s whole-run limit; earlier limits and failed attempts remain documented. Appendix A reports separate execution, checking, delivery, and canary costs. Unknown CPU/remote/human costs remain unknown, not zero.

Prespecified primary claims C1–C6 remain unrun; C7 is inconclusive on constructed diagnostics. Actual shared updates and structural feedback do not establish generalization to unseen source families. Independent fidelity is unmeasured. Optional author feedback was not supplied and is not required.

7 Related work

Mathematical autoformalization with large language models [Wu et al., 2022, Azerbayev et al., 2023, Jiang et al., 2024] motivates learned statement generation, typically into proof-assistant mathematics. Our source pipeline combines domain-specific parsing, structured teacher targets, adaptive representation heads, and checked compilation guidance for policy, code, and trace material. We do not report ProofNet, miniF2F, or multilingual math-autoformalization scores; those benchmarks were not run.

Retrieval-augmented and informal-to-formal proving [Jiang et al., 2022, 2023, Yang et al., 2023, First et al., 2023] integrate language models with hammers or whole-proof generation. Thor and Sledgehammer [Blanchette et al., 2013] reconstruct solver candidates in an originating ITP. Our Hammer and Leanstral routes are the corresponding implemented mechanisms, but sealed-PATH native checkers were unavailable, so we do not claim LeanDojo/Thor/Baldur performance.

Verified coding systems such as CompCert [Leroy, 2009], seL4 [Klein et al., 2009], and Dafny [Leino, 2010] supply the workshop’s coding-verification context: a specification, an implementation, and a machine-checked artifact. Our synthetic protected-write/audit example is a finite illustration of that use case, not a verified kernel, compiler, or program verifier, and those systems were not re-benchmarked here.

Learned perception plus symbolic execution is established in neural-symbolic reasoning [Yi et al., 2018]; here provenance, modal scope, and separate proof-feedback objectives are material. We do not claim to originate multi-logic interoperability: institutions formalize signatures, models, and satisfaction [Goguen and Burstall, 1992], while LogiKey combines normative logics by HOL embedding [Benzmüller et al., 2020]. Frame logic, event calculus, and translation validation are established foundations [Kifer et al., 1995, Kowalski and Sergot, 1986, Pnueli et al., 1998].

8 Limitations and conclusion

Independent human fidelity, agreement, and adjudication were not measured. Teacher bias, sample-memory shortcuts, lossy legacy translations, bounded checker coverage, and unactivated guidance remain limitations. Constructor availability is not semantic coverage; a CID binds encoding rather than interpretation. Proof, satisfiability, finite observation, and policy approval remain distinct.

The retained contribution is implemented A–E and T0–T5 conditions, completed three-seed shared updates and structural feedback, actual rejected promotion, finite conformance and policy–code–trace witnesses, and development checker receipts. All 1913-unit primary cells remain honestly unrun, unavailable, or unmeasured. Target-assisted reconstruction does not establish independent source fidelity or useful cross-view proof coverage on unseen material.

References

- Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023. URL <https://arxiv.org/abs/2302.12433>. Original PDF reference [2]. Verified against arXiv:2302.12433; no archival conference version used here.
- Christoph Benz Müller, Xavier Parent, and Leendert van der Torre. Designing normative theories for ethical and legal reasoning: LogiKey framework, methodology, and tool support. *Artificial Intelligence*, 287:103348, 2020. doi: 10.1016/j.artint.2020.103348. Original PDF reference [3]. Verified against the Artificial Intelligence journal record and DOI.
- Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C. Paulson. Extending Sledgehammer with SMT solvers. *Journal of Automated Reasoning*, 51(1):109–128, 2013. doi: 10.1007/s10817-013-9278-5. Related-work comparison for hammer reconstruction. Sledgehammer was not executed as a paper benchmark and was not re-benchmarked.
- Emily First, Markus N. Rabe, Yuriy Brun, and Talia Ringer. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1229–1241, 2023. doi: 10.1145/3611643.3616243. Related-work comparison only. Baldur was not executed as a paper benchmark and was not re-benchmarked.
- Joseph A. Goguen and Rod M. Burstall. Institutions: Abstract model theory for specification and programming. *Journal of the ACM*, 39(1):95–146, 1992. doi: 10.1145/147508.147524. Original PDF reference [4]. Verified against JACM 39(1) and DOI 10.1145/147508.147524.
- Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations*, 2023. Related-work comparison only. DSP was not executed as a paper benchmark and was not re-benchmarked.
- Albert Q. Jiang, Wenda Li, and Mateja Jamnik. Multi-language diversity benefits autoformalization. In *Advances in Neural Information Processing Systems*, volume 37, 2024. Original PDF reference [9]. Recovered from the supplied PDF. Closely related public preprint: Multilingual Mathematical Autoformalization, arXiv:2311.03755. No ProofNet/miniF2F numbers from that literature are reported as this paper’s results.
- Albert Qiaochu Jiang, Wenda Li, Szymon Tworowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. Thor: Welding hammers to integrate language models and automated theorem provers. In *Advances in Neural Information Processing Systems*, volume 35, 2022. Original PDF reference [10]. Verified against the NeurIPS 2022 Thor paper (authors, title, venue).
- Michael Kifer, Georg Lausen, and James Wu. Logical foundations of object-oriented and frame-based languages. *Journal of the ACM*, 42(4):741–843, 1995. doi: 10.1145/210332.210335. Original PDF reference [5]. Verified against JACM 42(4). The recovered PDF hyphenated “frame-based”; the journal title uses “Frame-Based”.
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, pages 207–220, 2009. doi: 10.1145/1629575.1629596. Verified-coding use-case comparison. seL4 was not re-benchmarked.
- Robert A. Kowalski and Marek J. Sergot. A logic-based calculus of events. *New Generation Computing*, 4:67–95, 1986. doi: 10.1007/BF03037383. Original PDF reference [6]. Verified against New Generation Computing 4 and DOI 10.1007/BF03037383.

- 364 K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic*
365 *for Programming, Artificial Intelligence, and Reasoning*, volume 6355 of *LNCS*, pages 348–370,
366 2010. doi: 10.1007/978-3-642-17511-4_20. Verified-coding use-case comparison. Dafny was
367 not re-benchmarked.
- 368 Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–
369 115, 2009. doi: 10.1145/1538788.1538814. Verified-coding use-case comparison. CompCert was
370 not re-benchmarked.
- 371 Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms*
372 *for the Construction and Analysis of Systems*, volume 1384 of *LNCS*, pages 151–166, 1998. doi:
373 10.1007/BFb0054170. Original PDF reference [7]. Verified against TACAS 1998 / LNCS 1384
374 and DOI 10.1007/BFb0054170.
- 375 Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Chris-
376 tian Szegedy. Autoformalization with large language models. In *Advances in Neural Information*
377 *Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2205.12615>. Original
378 PDF reference [1]. Verified against arXiv:2205.12615 and NeurIPS 2022 proceedings listing.
- 379 Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil,
380 Ryan J. Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented
381 language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
382 Related-work comparison only. LeanDojo was not executed as a paper benchmark and was not
383 re-benchmarked.
- 384 Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B. Tenen-
385 baum. Neural-symbolic VQA: Disentangling reasoning from vision and language understand-
386 ing. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL <https://arxiv.org/abs/1810.02338>. Original PDF reference [8]. Verified against NeurIPS 2018
387 and arXiv:1810.02338.

389 A Actual training and promotion execution

390 The completed common-profile run used 2 CPU cores, 64 GiB RAM, 64 processes, no network,
391 Torch intra-op threads 2/inter-op 1, and three unchanged seeds. T2’s native soft budget was 9000 s;
392 an independent host limit enclosed its full 10800 s stage, including evaluation and reload. T1 and
393 T3 were outside that stage but inside the 36000 s whole-run limit. This prospectively superseded the
394 historical 16 GiB/1800 s and residual 600 s profiles; it did not refund prior failed usage.

Table 6: Actual exclusive training-stage wall seconds. All seeds accepted one T2 epoch with five packed update families; T3 applied 256 structural-feedback updates each. No final-test outcome is implied.

Seed	T1	Complete T2	Complete T3
104729	24.724	2628.314	1809.538
130363	25.368	2646.454	1812.073
155921	24.906	2693.777	1841.133

395 All exclusive producer stages, including common preparation and feedback, sum to 13926.774 s.
396 The original checker then consumed 764.651 s and failed out of memory; the enclosing original
397 boundary lasted 14712.602 s with observed cgroup CPU at least 14681.662 s. The separately recovered retained-byte checker took 1265.157 s and 1258.410 process-CPU seconds; artifact delivery took 1299.967 s. These clocks are labeled by scope and must not be summed with their enclosing intervals. Known prior seed-attributed wall time was 1154.274/104.275/103.614 s respectively, with additional historical interruptions unknown. Two earlier memory failures and all original receipts remain retained. No hard cumulative-budget or zero-cost failure claim is made.

403 The three AF-013 fixed-canary runs each loaded one T3 state and retained 38 ordered native diagnostics (114 total), real export/promotion/ingestion/default-reset records, and exact cleanup. Native wall time totaled 1380.308 s and process CPU 1363.292 s; final cgroup CPU remains unknown. Each

406 reported all 38 identity ingestions and unchanged off/on/default-reset compiler cores, while actual
 407 learned-feature application stayed false. Every promotion returned no candidate with four block
 408 reasons: missing proof receipts, missing view-family weights, no resolved canonical contracts, and
 409 no guidance records meeting threshold. Matching digests and empty native metric maps are not
 410 measured efficacy or semantic fidelity.

411 The 69 source-bound feedback packets retained 60 checked, 7 timed-out, and 2 unsupported out-
 412 comes. Their 2606 eligible labels describe compiler structural contracts. Each T3 seed applied a
 413 bounded 256-label prefix under the same frozen procedure; no independent source-semantic labels
 414 or new fidelity aggregate were produced. T2 train/selection family cross-entropy is unchanged from
 415 T0; its cosine 1/MSE 0 uses the supplied target and is not a learned reconstruction gain.

416 B Packed training equations

417 The packed reconstruction term averages over embedding coordinates,

$$L_{\text{rec}} = \frac{1}{n} \sum_i \frac{1}{d} \|\hat{e}_i - e_i\|_2^2, \quad (11)$$

418 and masked family/view cross-entropy is globally normalized over eligible rows M ,

$$L_{\text{CE}} = -\frac{1}{n} \sum_{i \in M} \sum_j q_{ij} \log \text{softmax}(\ell_i)_j. \quad (12)$$

419 The inspected update is SGD-style,

$$\Theta \leftarrow \Theta - \eta \text{Clip}(\nabla_{\Theta} L). \quad (13)$$

420 These identities describe the audited packed-CPU path. They are not a claim that every configured
 421 head was enabled in a deployed checkpoint, and they are not held-out source fidelity.

422 C Conditional proof-transfer proposition

423 **Proposition 1 (conditional semantic consequence transfer).** Assume a translator τ , bridge as-
 424 sumptions B , and a relation R_B that is nonvacuous on the admitted source models of Γ . If Equations
 425 6–7 hold, premises are consistent in the relevant sense, and a sound target checker establishes
 426 $\tau(\Gamma) \models \tau(\phi)$, then $\Gamma \models_{L_s} \phi$. This is a paper-level conditional argument, not a theorem that every
 427 repository translator satisfies. Related-model existence, premise preservation, goal reflection, and
 428 checker identity must be discharged per supported fragment. The legacy TDFOL converter does not
 429 satisfy the contract.

430 D Historical recovery copy

431 The user-supplied PDF was reconstructed into editable pages under `pages/` and
 432 `source_recovery_staging/` with a discrepancy audit. That recovery copy preserves origi-
 433 nal wording, pending result fields, and historical claims; it is not this paper’s results narrative.
 434 Equations 1–13 were transcribed from that PDF and re-audited here for quantification, nonvacuity,
 435 and tense.

436 E Language-model, teacher-model, and asset disclosure

437 This appendix records methodology-essential model, teacher, and asset uses that were actually
 438 logged. It does not invent a version, a prompt, or an author review. Writing, editing, and for-
 439 matting assistance is disclosed for completeness; under the workshop checklist it does not by itself
 440 require a core-method declaration.

441 **E.1 Direct model-to-target (Arm A development)**

442 One development qualification call used the configured scientific gateway to
443 `https://api.x.ai/v1/chat/completions`. The requested and served model
444 was `grok-4.6` (system fingerprint `fp_e41c6060b2628547`) on 2026-09-12,
445 started 06:10:58 UTC and finished 06:11:09 UTC. The constructed source was a
446 guarded-write sentence (`AF027-DEV-Q1-protected-write`); the prompt digest is
447 `sha256:1d9b23b293a4d99932c23291242f90d2dad9d573c0c13e950beb3d7f2091fec8` and the completion digest
448 is `sha256:fb52b06935c4a867a2c17dc3e3784f417f49b2927ee47d99dcf5446964ab9ed8`. Usage was 821 prompt
449 tokens and 54 completion tokens. A Codex `gpt-5.6-terra` (reasoning effort high) fallback was
450 configured and was not invoked. The receipt sets `claim_admissible=false`; the call does not fill
451 Table 3.

452 **E.2 Embedding teacher and grouping encoder**

453 The lexical/paraphrase grouping encoder and later packed-CPU teacher
454 targets used `sentence-transformers/all-MiniLM-L6-v2` at revision
455 `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`, loaded locally (`local_files_only=true`,
456 `trust_remote_code=false`) with attention-mask mean pooling and L2 normal-
457 ization. The weight file `model.safetensors` is 90,868,376 bytes with SHA-256
458 `53aa51172d142c89d9012cce15ae4d6cc0ca6895895114379cacb4fab128d9db`. Earlier
459 AF-011 T0/T1 diagnostics used `mock:stable-sha256`; the completed AF-029 T0/T1/T2/T3
460 experiment used the MiniLM teacher above. The retained `claim_admissible=false` flag
461 excludes primary final-test/fidelity claims, not the actual completed training. Those updates do not
462 fill Table 4 held-out cells.

463 **E.3 Unavailable or unused model routes**

464 LLM plan nomination executed zero model calls. Leanstral proposal generation was not run; its
465 weights (67,135,119,264 bytes) exceeded the historical 16 GiB envelope for that unrun route; this
466 is distinct from the later 64 GiB CPU training profile. Hammer/Leanstral native reconstruction is
467 unavailable on the sealed PATH. Compiler-repair (AF-014) recorded `provider_calls=0`. No model
468 version is inferred for those unrun routes.

469 **E.4 Manuscript writing assistance**

470 The recovered source PDF stated that language-model assistance was used to research and draft
471 the manuscript and reference-example code, and that author verification had not occurred. This
472 AF-024 formatting pass used an implementation agent to apply the official template, complete the
473 questionnaire from logs, and compile the anonymous PDF. That writing and formatting use is not
474 a core experimental method, is not a substitute for author verification, and is not reported as a
475 measured result.

476 **E.5 Broader impacts**

477 A verified mapping from policy or code into a checked artifact can help reviewers notice missing
478 approvals, missing audits, or inconsistent premises before deployment. The same machinery can be
479 misused: an incorrect formalization of a legal or security obligation can be treated as authority; mon-
480 itoring queries can be aimed at people rather than at declared system behavior; and a solver success
481 can be presented as source-faithful proof. Mitigations retained here are abstention on unsupported
482 fragments, withholding holdout source bodies, refusing `sorry/admit` before any kernel claim, and
483 keeping teacher, reconstruction, and prover success distinct from independent source fidelity, which
484 remains unmeasured.

485 **E.6 Existing and new assets**

486 The legal/policy cohort is the upstream `justicedao/patent-legal-corpus` revision
487 `86c77cb650d30ee366983d1b2e25cd85e8c22e34`. Upstream record metadata declares re-
488 viewed public-domain U.S. government works and the dataset card declares CC0-1.0; those are

489 preserved declarations, not a new legal determination by this manuscript. Coding/implementation
490 shard rows include 72 upstream allow and 4024 needs_review license-risk labels and are not
491 admitted as natural evaluation data. MiniLM is the public sentence-transformers checkpoint named
492 above. New anonymous assets in this submission are the supplement ZIP (table regenerators, frozen
493 retained-claim inputs, S01–S40 hashes) plus hash-bound references for native packed-CPU states
494 that are not shipped. No author-hosted URL is published here.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the measured automated inventory (A–E/T0–T5 conditions, unrun 1913-unit cells, actual three-seed updates, rejected native promotion, constructed witnesses) and explicitly do not claim independent source fidelity or useful proof coverage on unseen material.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section “Limitations and conclusion” records uncollected independent labels, teacher bias, sample-memory shortcuts, lossy translations, unavailable native checkers, unactivated T4/E, and that reconstruction or prover success is not source gold.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The conditional transfer proposition states nonvacuity, premise preservation, goal reflection, and sound-checker assumptions (Eqs. 6–7; Appendix C) and is labeled a paper-level argument, not a machine-checked compiler theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Seeds 104729/130363/155921, frozen splits, and sealed-path commands are in the main text; the anonymous supplement regenerates Tables 3, 4, and 5 from frozen `summary.json`, retaining unrun cells rather than filling them.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The anonymized supplement documents regeneration commands and checksums; large packed-CPU states and MiniLM weights are hash-bound with anonymous access strategies, and private holdout bodies are withheld (Appendix E.6).

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The evaluation protocol, A–E/T0–T5 definitions, split counts, seeds, packed SGD-style update, and teacher identities appear in the main text; full receipts are in the supplement.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Primary 1913-unit cells are unrun or unmeasured, so no error bars are reported there; T1 reports exact seed-level teacher-cosine values, and the prespecified 95% cluster bootstrap is not imputed for missing cells.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The actual-execution appendix reports the amended 2-CPU/64 GiB profile, per-seed training stages, separate checker/delivery/canary costs, prior failures and remaining unknowns. Historical source usage is 481.586 s; nested clocks and phase summaries are not double-counted.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The study uses upstream-declared public-domain U.S. legal/policy text, withholds private holdout bodies, does not collect human-subject data, and does not fabricate annotations.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix E.5 records potential verification benefits and harms from treating incorrect legal or policy formalizations as authority, including misuse of monitoring queries and solver success presented as source-faithful proof.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No high-risk generative model or scraped internet image dump is released; holdout source bodies remain private and MiniLM is a public encoder checkpoint (Appendix E.6).

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

760 Answer: [\[Yes\]](#)

761 Justification: Appendix E.6 records upstream CC0-1.0 / public-domain-U.S.-government

762 declarations for the legal corpus, the MiniLM model id and revision, and that 4024 coding-

763 shard license-risk rows remain needs_review and are not admitted as natural data.

764 Guidelines:

- 765 • The answer [\[N/A\]](#) means that the paper does not use existing assets.
- 766 • The authors should cite the original paper that produced the code package or dataset.
- 767 • The authors should state which version of the asset is used and, if possible, include a
- 768 URL.
- 769 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 770 • For scraped data from a particular source (e.g., website), the copyright and terms of
- 771 service of that source should be provided.
- 772 • If assets are released, the license, copyright information, and terms of use in the pack-
- 773 age should be provided. For popular datasets, paperswithcode.com/datasets has
- 774 curated licenses for some datasets. Their licensing guide can help determine the li-
- 775 cense of a dataset.
- 776 • For existing datasets that are re-packaged, both the original license and the license of
- 777 the derived asset (if it has changed) should be provided.
- 778 • If this information is not available online, the authors are encouraged to reach out to
- 779 the asset’s creators.

780 **13. New assets**

781 Question: Are new assets introduced in the paper well documented and is the documenta-

782 tion provided alongside the assets?

783 Answer: [\[Yes\]](#)

784 Justification: The anonymous supplement documents regeneration, checksums, and S01–

785 S40 hashes; new code and large checkpoints beyond that package are hash-bound rather

786 than silently omitted (Appendix E.6).

787 Guidelines:

- 788 • The answer [\[N/A\]](#) means that the paper does not release new assets.
- 789 • Researchers should communicate the details of the dataset/code/model as part of their
- 790 submissions via structured templates. This includes details about training, license,
- 791 limitations, etc.
- 792 • The paper should discuss whether and how consent was obtained from people whose
- 793 asset is used.
- 794 • At submission time, remember to anonymize your assets (if applicable). You can
- 795 either create an anonymized URL or include an anonymized zip file.

796 **14. Crowdsourcing and research with human subjects**

797 Question: For crowdsourcing experiments and research with human subjects, does the pa-

798 per include the full text of instructions given to participants and screenshots, if applicable,

799 as well as details about compensation (if any)?

800 Answer: [\[N/A\]](#)

801 Justification: No crowdsourcing or human-subjects experiment was conducted; the 100-

802 unit candidate-blind sample remains blank and is not reported as agreement.

803 Guidelines:

- 804 • The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research
- 805 with human subjects.
- 806 • Including this information in the supplemental material is fine, but if the main contri-
- 807 bution of the paper involves human subjects, then as much detail as possible should
- 808 be included in the main paper.
- 809 • According to the NeurIPS Code of Ethics, workers involved in data collection, cura-
- 810 tion, or other labor should be paid at least the minimum wage in the country of the
- 811 data collector.

812 **15. Institutional review board (IRB) approvals or equivalent for research with human**
813 **subjects**

814 Question: Does the paper describe potential risks incurred by study participants, whether
815 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
816 approvals (or an equivalent approval/review based on the requirements of your country or
817 institution) were obtained?

818 Answer: [N/A]

819 Justification: No human-subjects research was performed, so no IRB or equivalent review
820 applies.

821 Guidelines:

- 822 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
823 with human subjects.
- 824 • Depending on the country in which research is conducted, IRB approval (or equivalent)
825 may be required for any human subjects research. If you obtained IRB approval,
826 you should clearly state this in the paper.
- 827 • We recognize that the procedures for this may vary significantly between institutions
828 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
829 guidelines for their institution.
- 830 • For initial submissions, do not include any information that would break anonymity
831 (if applicable), such as the institution conducting the review.

832 **16. Declaration of LLM usage**

833 Question: Does the paper describe the usage of LLMs if it is an important, original, or
834 non-standard component of the core methods in this research? Note that if the LLM is used
835 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
836 scientific rigor, or originality of the research, declaration is not required.

837 Answer: [Yes]

838 Justification: Appendix E records served grok-4.6 as the Arm A development model-to-
839 target on 2026-09-12, MiniLM revision 1110a243 . . . as a grouping/teacher encoder, and
840 zero Llantrai/plan-nomination calls; writing-only assistance is disclosed there and is not
841 experimental evidence.

842 Guidelines:

- 843 • The answer [N/A] means that the core method development in this research does not
844 involve LLMs as any important, original, or non-standard components.
- 845 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
846 not be described.